ChatGPT

Literature Review on AI-Powered Blast Radius Analysis

We surveyed key research (2015–2026) across the 23 topic areas listed. Each **relevant paper** is summarized with its bibliographic details and key content, and we note how it could inform our system's components. (Citations are given to validated sources for claims.)

Change Impact Analysis

- **Kretsou et al. (2021)** – *Change Impact Analysis: A Systematic Mapping Study*【69†L84-L92】. **Authors:** M. Kretsou, E.-M. Arvanitou, et al. Published in J. Syst. Softw. (2021). DOI:10.1016/j.jss.2020.110892. **PDF:** (ScienceDirect) – no free PDF. **ArXiv/GitHub:** n/a. **Citations:** ~50. **Area:** CIA (survey). **Problem:** Understand the state of CIA research and metrics. **Core Idea:** Mapped 111 papers on CIA, categorizing methods and quantified factors (instability, change-proneness, amount of change, changeability)【69†L84-L92】. **Architecture:** Systematic literature review. **Algorithms/Models:** N/A. **Data:** Academic papers (2010–2020). **Evaluation:** Qualitative mapping; identified over- and under-studied areas. **Strengths:** Broad coverage; distilled core metrics (e.g. instability)【69†L84-L92】. **Weaknesses:** No empirical evaluation on actual code, only literature analysis. **Limitations:** Focused on academic studies, may omit industrial practices. **Relevance:** Provides context for CIA concepts (metrics, benefits) useful for our Requirements and Impact components. **Action:** *Read* – adopt its insights on CIA metrics and practical benefits, adapt by integrating those metrics into our blast radius scoring.

- **Abdurhman & Lee (2023)** – *FCP2Vec: Deep Learning-Based Approach to Software Change Prediction…* Appl. Sci. 13(11):6453 (2023). **PDF:** MDPI. **ArXiv:** n/a. **GitHub:** n/a. **Citations:** ~6. **Area:** CIA via co-change. **Problem:** Predict which files co-change (impact scope) in large code. **Core Idea:** Treats a changelog as sequences (skip-gram Word2Vec) to embed files ("file commit patterns"), then predicts co-changes via nearest neighbors【7†L67-L76】. **Architecture:** Unsupervised embedding (skip-gram) + kNN. **Algorithms/Models:** Word2Vec (skip-gram), kNN for recommendation. **Data:** 3 open-source projects' commit logs (Git history for Vuze, Spring, Elasticsearch)【7†L67-L76】. **Evaluation:** Hit Ratio@K (recall of co-changed files). Achieves ~21% better recall than traditional dependency networks【7†L73-L77】. **Strengths:** Scales to large codebases, learns from history automatically. **Weaknesses:** Requires rich historical data; only file-level (not semantic). **Limitations:** Evaluated on Java projects only; may not capture semantics of changes. **Relevance:** Useful for our Repository Analysis / Impact component (predicting affected code from PR). **Action:** *Adapt* – use its embedding idea for recommending potentially affected components from commit diffs.

Requirements Traceability

- **Wang et al. (2024)** – *An empirical study on SOTA methods for requirements-to-code traceability link recovery*【24†L76-L84】. J. King Saud Univ. Comp. Info. Sci., 36(6):102118 (2024). **DOI:**10.1016/j.jksuci.

2024.102118. **PDF:** (Open access via ScienceDirect). **Citations:** 0. **Area:** Traceability (requirements-code). **Problem:** Lack of a benchmark comparing RC traceability methods. **Core Idea:** Systematic literature review (2018–2023) of methods for linking requirements to code, with an experimental replication on 13 datasets【24†L76-L84】. **Architecture:** Review+empirical evaluation framework. **Algorithms:** Evaluated 6 SOTA methods (mostly IR-based and ML-based). **Models:** IR (CRT, TAROT, Fine-grained TLR, POS+IR), ML (Random Forest). **Data:** 13 standard traceability datasets (COEST, large industrial ones). **Evaluation:** Replicated methods, measured precision/recall. Findings: CRT, TAROT, FTLR work well on medium data; ConPos and TAROT good on large sets; RandomForest strong across the board【24†L79-L84】. **Strengths:** First comparative benchmark in RC-TLR, open resources provided. **Weaknesses:** Only covers methods up to 2023; evaluation limited to preset datasets. **Limitations:** Focused on IR and basic ML, not neural methods. **Relevance:** Guides linking PRDs to code (our requirement linking component). **Action:** *Read* – adapt insights on effective traceability features (TF-IDF, POS, etc.) and consider implementing TAROT or RandomForest as part of our link recovery.

Software Knowledge Graphs

- **Zou et al. (2021)** – *Software knowledge graph construction and Q&A technology based on big data*. 大数据 (Big Data) 7(1):2021002 (Jan 2021). **Authors:** Y. Zou, M. Wang, B. Xie, Z. Lin. **DOI:**10.11959/j.issn.2096-0271.2021002. **PDF:** (no direct link; Chinese journal). **Citations:** unknown. **Area:** Software KGs. **Problem:** Building rich software project KGs from heterogeneous sources (code, docs, Q&A). **Core Idea:** Proposes a *code-centric knowledge model* and a two-layer plugin framework to construct KGs and integrate Q&A【26†L103-L112】. **Architecture:** Two-layer pipeline (data extraction plugins + Q&A service). **Algorithms:** Not specified (rule-based extraction inferred). **Models:** Ontology model defined in the paper. **Data:** Multi-source: code, issue reports, mailing lists, Q&A docs. **Evaluation:** Deployed KG in Apache projects and enterprises (qualitative). **Strengths:** Holistic integration of many data sources; deployed in real projects. **Weaknesses:** Details on extraction methods are sparse. **Limitations:** Lacks public dataset or code. **Relevance:** Offers a model for combining code and project artifacts into a KG (our Knowledge Graph component). **Action:** *Adapt* – use its code-centric ontology and plugin idea to incorporate PRDs and UI artifacts into our KG.

- **Wang et al. (2023)** – *Application of knowledge graph in software engineering field: A systematic literature review*【35†L152-L160】【69†L84-L92】. Inf. Softw. Technol. 164:107327 (Dec 2023). **Authors:** L. Wang, C. Sun, C. Zhang. **DOI:**10.1016/j.infsof.2023.107327. **PDF:** (ScienceDirect). **Citations:** 0. **Area:** Software KGs (survey). **Problem:** No comprehensive survey on KGs in SE. **Core Idea:** SLR of 100+ papers (since 2017) on KGs for SE【35†L152-L160】. Answered: “What knowledge is modeled?”, “How built/optimized?”, “Which methods & applications?”【35†L152-L160】. **Architecture:** Literature mapping + analysis of content. **Algorithms:** N/A. **Models:** Summary of KGs and ontologies used across SE tasks. **Data:** 55 key papers selected. **Evaluation:** Qualitative insights; graphs (not provided to us). **Strengths:** Thorough review; identified research trends and gaps. **Weaknesses:** Chinese context, but published in English. **Limitations:** Survey only (no new method). **Relevance:** Summarizes state of Software KGs (background for Knowledge Graph Construction). **Action:** *Read* – use its taxonomy of KG applications to identify relevant tasks (e.g., KG used in testing, vulnerability). Adapt its identified best practices (e.g. semantic relations used).

Code Property Graphs (CPG)

- **Saimbhi (2025)** – *Enhancing Software Vulnerability Detection Using CPGs and CNNs*【53†L33-L41】. In ICCCIT 2025. **Author:** A. S. Saimbhi. **DOI:**10.1109/ICCCIT62592.2025.10928033. **ArXiv:** 2503.18175. **PDF:** arXiv. **Citations:** 0. **Area:** CPG / static analysis. **Problem:** Improve vulnerability detection beyond traditional static analysis. **Core Idea:** Builds Code Property Graphs (AST+CFG+DDG) for functions, and feeds them to CNNs specialized for graph data【53†L33-L41】. **Architecture:** (1) Transform code→CPG; (2) Train CNN on CPG data. **Algorithms/Models:** Graph-based convolutional neural networks (GCNN). **Data:** Newly created dataset of functions labeled by vulnerability type (from open-source). **Evaluation:** Accuracy on function-level vulnerability detection tasks (ICCCIT paper format suggests ROC or accuracy, but details not in abstract). **Strengths:** Bridges graph modeling of code and deep learning. **Weaknesses:** Likely high computational cost; single author preprint (not community-validated). **Limitations:** Focuses only on security vulnerabilities, not general CIA. **Relevance:** Technique could be adapted for analyzing code graph structure in impact analysis (Repository Analysis). **Action:** *Consider* – investigate using CPG + GNN for structural code similarity or risk scoring (if applicable to our backend component).

Graph-based Software Engineering / Ontologies / Knowledge Construction

- **RepoDoc: Xu et al. (2026)** – *RepoDoc: A Knowledge Graph-Based Framework to Automatic Documentation Generation and Incremental Updates*【32†L83-L91】. *ArXiv* 2604.26523v1 (29 Apr 2026). **Authors:** D. Xu, M. Liu, X. Wang, J. Zhong, Z. Zheng. **DOI:** (ArXiv 2604.26523). **PDF:** ArXiv. **GitHub:** SYSUSELab/RepoDoc. **Citations:** 0 (new). **Area:** Knowledge graphs / documentation (multi-topic: Knowledge Graph, Retrieval, Reasoning). **Problem:** Automated, up-to-date documentation at repository scale. **Core Idea:** Constructs a **Repository Knowledge Graph (RepoKG)** of code entities/relations; clusters modules; uses agent-based LLM queries to generate modular documentation; and propagates changes via graph traversal【32†L83-L91】. **Architecture:** Three-phase pipeline + incremental update: (1) **KG Construction** from AST (entities and relations)【32†L83-L91】; (2) **Module Clustering** by graph-based functional cohesion; (3) **LLM Agent Generation** using graph queries (retrieving only relevant code)【32†L83-L91】. Plus **Semantic Impact Propagation** for updates. **Algorithms/Models:** AST parsing, graph clustering, topological sort (for update consistency), LLM (DeepSeek V3.2) as generator agents. **Data:** 24 diverse repos (Python, JS/TS, Java, C#, C/C++, PHP) in evaluation【32†L93-L100】. **Evaluation:** Compared against doc baselines (RepoAgent, CodeWiki). Metrics: documentation **coverage** (APIs documented), completeness, time/token usage. RepoDoc improved coverage by 32.5% and completeness by 10.4%, with 3× faster generation and 85% fewer tokens【32†L93-L100】; incremental updates 73% faster, 77% fewer tokens, 10.2% higher recall. **Strengths:** Combines static analysis (RepoKG) with LLMs for structured doc generation; addresses incremental updates explicitly. **Weaknesses:** Depends on LLM backend; metrics specific to docs, not directly to code behavior. **Limitations:** Early-stage (ArXiv), but code is available. **Relevance: Crucial** – demonstrates how a knowledge graph can underlie both understanding code/UI and generating text (report generation and documentation). Its impact propagation can inspire our Reasoning component. **Action:** *Adopt/Extend* – adopt the RepoKG concept for linking code modules and adopt its impact propagation algorithms for our Blast Radius report. Use its GitHub tools if possible.

- **Han et al. (2025)** – *Retrieval-Augmented Generation with Graphs (GraphRAG)*【41†L116-L124】【41†L129-L137】. *ArXiv* 2501.00309v2 (Jan 2025). **Authors:** H. Han, Y. Wang, H. Shomer, et al. **DOI:** 10.48550/arXiv.2501.00309. **PDF:** ArXiv. **GitHub:** GraphRAG repo. **Citations:** 0 (new). **Area:** Graph retrieval + RAG. **Problem:** Integrating graph-structured data as the knowledge source in RAG frameworks. **Core Idea:** Surveys and proposes a **GraphRAG** framework: defines components (retriever, organizer, generator) specialized for graph data【41†L122-L130】. Reviews GraphRAG for multiple graph types (text, knowledge, social, planning graphs)【41†L122-L130】. **Architecture:** Conceptual (survey): query processor → graph retriever → organizer → generator. **Algorithms/Models:** Discusses graph embedding (GraphSAGE, GNN), query planning, LLM integration; no single model. **Data:** Domains: Infrastructure graphs, scene graphs, etc. **Evaluation:** None (survey). **Strengths:** Comprehensive overview of how to combine KGs/graph DBs with LLM retrieval, including multi-hop reasoning. **Weaknesses:** No empirical results (conceptual survey). **Limitations:** Very new; primarily high-level. **Relevance:** Provides the theory of using a **Knowledge Graph as a retrieval source for an LLM (i.e. GraphRAG)**. This directly inspires using the Product KG to feed context to LLM for explanation. **Action:** *Read* – adopt its components (Retriever, Organizer, Generator) design for our Graph Retrieval and Report Generation. Adapt its insights on graph-based retrieval to query our KG effectively.

- **Lewis et al. (2020)** – *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*【43†L39-L48】. NeurIPS 2020. **Authors:** P. Lewis, E. Perez, et al. **DOI:**10.48550/arXiv.2005.11401. **PDF:** ArXiv / NeurIPS. **Citations:** ~2000 (high-impact). **Area:** RAG (text-based). **Problem:** LLMs lack up-to-date or precise world knowledge; hallucinate. **Core Idea:** Introduces *RAG* models that combine a seq2seq LLM (parametric) with a non-parametric memory (external retriever over Wikipedia) for generation【43†L39-L48】. Two variants: retrieve same passages for all tokens vs. per-token retrieval. **Architecture:** Seq2seq Transformer (BART) + Dense Passage Retriever indexing Wikipedia. At generation, retrieve relevant docs to condition LLM. **Algorithms/Models:** BART / DPR. **Data:** Wikipedia + standard QA tasks (Natural Questions, TriviaQA, etc.). **Evaluation:** Achieved SOTA on open-domain QA; found RAG produces more factual, diverse text than pure LLM【43†L39-L48】. **Strengths:** Pioneering RAG approach, open-sourced. **Weaknesses:** Retrieval limited to text corpus; not graph-structured. **Limitations:** Large compute; retrieval latency. **Relevance:** Conceptual foundation for *Retrieval-Augmented Generation*. Suggests how to augment LLM with evidence (in our case, from the product KG). **Action:** *Adapt* – use the idea of retrieving contextual evidence (via knowledge graph) for justifying our blast radius explanations.

Knowledge Graph Construction / Ontology / Knowledge Fusion / Autonomous Crawling

- **Peshevski et al. (2025)** – *AI Agent-Driven Framework for Automated Product Knowledge Graph Construction in E-Commerce*【45†L30-L39】. First GOBLIN Workshop on KG Tech (2025). **Authors:** D. Peshevski, R. Stojanov, D. Trajanov. **ArXiv:** 2511.11017 (Nov 2025). **DOI:**10.48550/arXiv.2511.11017. **PDF:** ArXiv. **Citations:** 0. **Area:** Autonomous agents / KG construction. **Problem:** Manually building KGs for product data is slow and schema-bound. **Core Idea:** Uses *LLM-based agents* to fully automate KG construction from unstructured product descriptions (e.g. e-commerce data)【45†L30-L39】. Three-agent pipeline: Ontology creation/expansion, ontology refinement, KG population【45†L30-L39】. No predefined schema is needed; agents handle semantics via LLM reasoning. **Architecture:** Multi-agent LLM framework (prompt-based): one agent designs ontology (classes/relations), another

refines schema, a third populates KG from text. **Algorithms/Models:** GPT-style LLMs (not specified); agent orchestration. **Data:** Real-world dataset of air conditioner product descriptions. **Evaluation:** Measures coverage (97% of relevant properties captured) and low redundancy【45†L39-L47】. **Strengths:** Scalability, semantic coherence without hard rules, minimal schema overhead. **Weaknesses:** Depends on LLM correctness; domain limited (only products). **Limitations:** Preliminary (workshop), no code public (that we saw). **Relevance:** Shows how autonomous agents (LLMs) can crawl and extract information into a KG (Crawling, KG Construction, Ontology Design). Suggests using LLMs to populate our Product KG from docs. **Action:** *Consider* – apply agent-based KG population to PRDs and UI docs (we may adapt its agent prompts). Treat it as a model to fuse heterogeneous docs into our graph (knowledge fusion).

- **Aleti et al. (2026)** – *Trustworthy AI Software Engineers*【64†L16-L24】. *ArXiv* 2602.06310v1 (Feb 2026). **Authors:** A. Aleti, B. Ray, R. Hoda, S. Chen. **PDF:** ArXiv. **Citations:** 0. **Area:** AI Agents for SE (vision/trust). **Problem:** Defining the role and trustworthiness of AI agents as software engineers. **Core Idea:** Presents a vision of AI coding agents as team members in SE; identifies dimensions of trustworthiness (technical quality, transparency, fairness, epistemic humility, ethical alignment)【64†L19-L28】. **Architecture:** Conceptual discussion (no system built). **Algorithms:** N/A. **Data:** N/A. **Evaluation:** N/A (conceptual). **Strengths:** Frames critical questions about AI agents replacing/cooperating with humans; calls for "ethics-by-design" in SE tools. **Weaknesses:** No empirical validation. **Limitations:** Broad; not specific to blast radius but informs design. **Relevance:** Highlights that any autonomous AI module (e.g. for impact analysis) must be trustworthy and auditable – which motivates our Explainable AI component. **Action:** *Read* – use its trustworthiness dimensions to guide how we present blast radius evidence (transparency, provenance). It suggests rigorous evaluation strategies for AI code assistants.

GUI/DOM Understanding

- **Tang et al. (2025)** – *A Survey on (M)LLM-Based GUI Agents*【47†L72-L80】. *ArXiv* 2504.13865v2 (Apr 2025). **Authors:** F. Tang, H. Xu, et al. (Zhejiang Univ/Microsoft Research Asia). **PDF:** ArXiv. **Citations:** 0 (new). **Area:** GUI Agents (multimodal LLM). **Problem:** Automating complex GUI tasks (web/mobile/desktop) with AI. **Core Idea:** Surveys the rise of AI-driven GUI agents, analyzing their architectures and components【47†L72-L80】. Identifies four components: (1) **Perception:** multimodal understanding of UI (text+vision); (2) **Exploration:** building knowledge of UI via internal/external info; (3) **Planning:** reasoning for task execution; (4) **Interaction:** generating actions with safety controls【47†L72-L80】. **Architecture:** N/A (survey). **Algorithms:** Discusses OCR, DOM parsing, LLMs, RL for GUI tasks. **Data:** Various GUI benchmarks (not in abstract). **Evaluation:** Categorizes and critiques existing evaluation strategies. **Strengths:** Comprehensive taxonomy; highlights LLMs revolutionizing interface automation. **Weaknesses:** No new model. **Limitations:** Survey covers wide domain; our focus is narrower (DOM understanding). **Relevance:** The *Perception* component is relevant to parsing UIs/dom (for User Experience linking). Insights on combining text and visual cues can help interpret UI pages and DOM elements in our product graph. **Action:** *Adapt* – use its identified perception techniques (e.g. text+vision parsing) when ingesting UI docs or screenshots. Its discussion of knowledge operation and planning may inspire how to "traverse" user flows in the KG.

Static Code Analysis

- **Salih & Sarhan (2025)** – *A Systematic Survey on LLMs for Static Code Analysis*【49†L74-L82】. ARO (Koya Univ.) 13(1): [2025]. **Authors:** H.A. Salih, Q.I. Sarhan. **DOI:**10.14500/aro.12082. **PDF:** Journal. **Citations:** 0. **Area:** Static analysis + LLM. **Problem:** Explore how LLMs can aid static code analysis tasks. **Core Idea:** Reviews recent works applying LLMs (GPT, CodeBERT, etc.) to detect bugs, vulnerabilities, and code smells in source code【49†L74-L82】. **Architecture:** Literature survey. **Algorithms:** Discusses various LLM applications (fine-tuning, few-shot), metrics and tools. **Data:** Published papers on LLM for code analysis. **Evaluation:** Tabulates tasks, advantages, limitations of each approach. **Strengths:** Identifies trends (LLMs handle semantics and context better than traditional static analyzers). **Weaknesses:** Broad summary, not a dataset or new technique. **Limitations:** Survey oriented; quickly outdated in fast-moving field. **Relevance:** Conveys that modern static analysis can leverage LLMs for explainable insights. This supports our need to make analysis explainable (by using LLM interpretations). **Action:** *Read* – note which static analysis tasks (e.g. vulnerability, type errors) LLMs excel at. Adapt those methods to interpret code changes in pull requests.

Repository Mining

- **Romero-Arjona et al. (2026)** – *The Rise of Language Models in Mining Software Repositories: A Survey*【56†L68-L77】. ACM JACM 37(4):111 (Aug 2026). **Authors:** M. Romero-Arjona, S. Barakat, A.B. Sánchez, S. Segura (Sevilla Univ.). **DOI:** (not yet published in this text). **PDF:** ArXiv. **Citations:** 0. **Area:** MSR + LLM (survey). **Problem:** How Transformer LLMs are changing Mining Software Repositories. **Core Idea:** Comprehensive survey of 85 papers on applying LMs/LLMs to MSR tasks【56†L68-L77】. Examines usage trends, data sources, tasks (commit classification, issue analysis, code summary, etc.) and proposes a taxonomy. **Architecture:** Survey methodology. **Algorithms:** Mostly Transformer-based (BERT, GPT variants, CodeBERT). **Data:** Open-source repositories (GitHub, etc.), commit messages, code, issues. **Evaluation:** Meta-analysis (no new experiments). **Strengths:** Up-to-date (2023–2026), covers broad trend of LLM in SW engineering. **Weaknesses:** Overviews only; does not give domain-specific guidance. **Limitations:** Only published works (may miss industry practices). **Relevance:** Highlights cutting-edge ML methods on repository data. Informative for how to preprocess and query repo content for our Analysis module (Repository Mining component). For example, shows feasibility of natural language processing on commit logs, which relates to FCP2Vec and PR analysis. **Action:** *Read* – use its taxonomy to align our Repository Mining with popular tasks (e.g. commit message classification, code summarization). Consider its observations about reproducibility and data preparation.

Explainable AI

- **Arora et al. (2025)** – *Explainable AI Techniques for Software Development Lifecycle: A Phase-specific Survey*【59†L87-L95】. (In press; arXiv 2505.07058). **Authors:** L. Arora, S.S. Girija, A. Raj, et al. (Google). **DOI:**10.48550/arXiv.2505.07058. **PDF:** ResearchGate/arXiv. **Citations:** 0 (new). **Area:** XAI in SE (survey). **Problem:** AI models in software engineering are black boxes; lack XAI. **Core Idea:** Surveys XAI methods (LIME, SHAP, concept-based, attention, counterfactuals, examples) applied across SDLC phases (requirements through maintenance)【59†L87-L95】. Reports most research is in maintenance (68% XAI work) and gaps in earlier phases. **Architecture:** Survey. **Algorithms:** Enumerates explanation techniques (model-agnostic and model-specific). **Data:** Literature (33

references). **Evaluation:** Qualitative discussion of technique suitability by phase. **Strengths:** First broad survey of XAI for software engineering. **Weaknesses:** No new experiments or system. **Limitations:** Quickly outdated in fast ML field. **Relevance:** Directly informs our **Explainable AI** component: which XAI methods to use for linking PR changes to outputs. For example, counterfactual or attention explanations may be used to highlight why a component is in the blast radius. **Action:** *Read* – borrow its mapping of XAI techniques to SE tasks. Adapt methods (e.g. use LIME or SHAP on any ML predictions we use, ensure we provide concrete evidence/support for our report).

Human-in-the-Loop AI

- **Wu et al. (2022)** – *A Survey of Human-in-the-Loop for Machine Learning*【61†L93-L101】. Future Gener. Comp. Syst. 135:364 (Oct 2022). **Authors:** X. Wu, L. Xiao, et al. **DOI:**10.1016/j.future.2022.05.014. **PDF:** ScienceDirect Preview. **Citations:** 560 (by 2025). **Area:** HITL in ML. **Problem:** Pure ML often requires too much data/training; incorporating human knowledge can improve efficiency and accuracy. **Core Idea:** Surveys HITL approaches from a data perspective: categorizes works into (1) data-centric (human annotates/improves data), (2) model-centric (human intervenes in training), (3) system design (interactive ML systems)【61†L93-L101】. **Architecture:** Survey. **Algorithms:** Not new, but summarizes various interactive learning frameworks, active learning, weak supervision. **Data:** Numerous references. **Evaluation:** Discussion of strengths/weaknesses of HITL in CV, NLP, etc. **Strengths:** Comprehensive categorization (data vs. model vs. system). **Weaknesses:** Broad ML focus, not SE-specific. **Limitations:** Survey – no new ML model. **Relevance:** Suggests our system could involve users (developers) in the loop for validating or guiding analysis (e.g. by giving feedback on identified blast radius). Highlights how to integrate user feedback to refine models. **Action:** *Consider* – design our report-generation interface to allow developer input (e.g. confirming impacts), and revisit findings; adapt active learning to improve impact predictions.

Regression Test Selection

- **Plyusnin et al. (2025)** – *Targeted Test Selection (T-TS) Approach in CI*【66†L50-L59】. *ArXiv* 2509.10279v1 (Sept 2025). **Authors:** P. Plyusnin, A. Antonov, et al. (T-Technologies). **DOI:** (arXiv 2509.10279). **PDF:** ArXiv. **Citations:** 0. **Area:** Regression Test Selection (RTS). **Problem:** Running full test suites on every CI commit is costly. **Core Idea:** Proposes **Targeted Test Selection (T-TS)** – an ML method that represents a commit as a "bag-of-words" of changed files plus cross-file features, avoiding heavy coverage maps【66†L50-L58】. Deployed in industry. **Architecture:** ML pipeline: feature extraction (file changes, code analysis), train classifier to predict which tests to run. **Algorithms:** Gradient boosting (likely, or other classifier – not detailed in abstract). **Models:** ML classifier (internally proprietary). **Data:** Both open-source and large private repositories. **Evaluation:** In production CI: T-TS ran only ~15% of tests, cut test time by ~85%, and still detected >95% of failures【66†L55-L60】. **Strengths:** Highly practical; significant savings in a real system. **Weaknesses:** Proprietary (full details not published), unknown generality. **Limitations:** Specific to the T-TS implementation (not open). **Relevance:** Demonstrates ML-based change impact on testing (a form of blast radius analysis for tests). It shows the feasibility of predicting affected components (tests here). **Action:** *Adapt* – use its data representation idea (commit as changed-file tokens) for modeling blast impact. If available, consider contacting authors for more details; otherwise use as inspiration for our impact scoring.

Summary of Papers by Component

We group the above (and related) works by architecture component:

- **Crawling / Data Collection:** Peshevski (2025) shows AI agents for **autonomous crawling and KG population** from product pages【45†L30-L39】. (Also Wu's HITL survey implies combining human web queries.)
- **Documentation Parsing:** RepoDoc (Xu et al.) provides **RepoKG construction from code** and LLM-based documentation generation【32†L83-L91】, which parallels parsing PRDs into KG.
- **Repository Analysis:** Romero (2026) and FCP2Vec (Abdurhman 2023) apply LMs and embeddings on repo artifacts to find patterns (commit/bug prediction)【7†L67-L76】【56†L68-L77】.
- **Knowledge Fusion / Ontology:** Peshevski (2025) and Zou (2021) address **KG construction/fusion** – Peshevski merges product data via agents【45†L30-L39】; Zou builds a code-centric KG for project data【26†L103-L112】.
- **Identity Resolution:** (No direct paper found. Standard ER techniques from data science might be used to merge duplicated artifacts in the KG.)
- **Knowledge Graph:** RepoDoc, Zou, Wang (2023), Peshevski all describe building software KGs【26†L103-L112】【32†L83-L91】【45†L30-L39】.
- **Graph Retrieval:** Han (2025) details the **GraphRAG** framework【41†L122-L130】, teaching how to retrieve from KGs; Lewis (2020) provides the baseline RAG.
- **Reasoning:** Aleti (2026) and Arora (2025) highlight *trust and explainability* issues in AI (ensuring reasoning is auditable)【64†L19-L28】【59†L87-L95】. GUI Agents (Tang 2025) cover *planning* in interface tasks【47†L72-L80】.
- **Report Generation:** RepoDoc uses LLM agents to generate and update documentation【32†L83-L91】; Arora (2025) emphasizes producing explainable decisions (which our report must do)【59†L87-L95】.

Comparison Table of Papers

Paper (Year)	Area(s)	Approach	Data	Key Result
Kretsou et al. (2021)	CIA (survey)	Systematic mapping of 111 papers【69†L84-L92】	lit review	Enumerates CIA metrics (instability, etc.)
Abdurhman & Lee (2023)	CIA / Co-change	Skip-gram embeddings (FCP2Vec)【7†L67-L76】	3 Java repos	+21% recall vs dependency network
Wang et al. (2024)	Traceability (REQ→Code)	SLR + experiments (IR & ML)【24†L76-L84】	13 datasets	RandomForest & TAROT work best
Zou et al. (2021)	SW Knowledge Graph	Code-centric KG model + plugin framework【26†L103-L112】	Apache repos	Deployed in practice
Wang et al. (2023)	SW Knowledge Graph	SLR of 55 papers	lit review	Taxonomy of KG methods in SE

Paper (Year)	Area(s)	Approach	Data	Key Result
Xu et al. (2026)	Doc Generation / KG	RepoKG + LLM agents【32†L83-L91】	24 repos	+32% API coverage; 3× speed up【32†L93-L100】
Han et al. (2025)	GraphRAG	Survey & framework	lit review	GraphRAG conceptual model
Lewis et al. (2020)	RAG	Dense retriever + BART	Wikipedia/ Q&A	SOTA QA; factual generation
Peshevski et al. (2025)	KG Construction/ Agents	LLM agents for ontology/KG【45†L30-L39】	Product descriptions	97% property coverage【45†L39-L47】
Tang et al. (2025)	GUI/DOM Agents	Survey of LLM-based GUI agents【47†L72-L80】	lit review	Defines perception & planning components
Salih & Sarhan (2025)	Static Analysis + LLMs	Survey of LLMs for code	lit review	XAI can aid traditional static analysis
Saimbhi (2025)	CPG + Vulnerability	CPG + CNN (GraphConvNet)【53†L33-L41】	New vuln. dataset	Basis for graph-based vulndet
Romero-Arjona et al. (2026)	MSR + LLM	SLR of 85 papers【56†L68-L77】	lit review	Taxonomy of LLMs applied to MSR
Arora et al. (2025)	XAI in SE	Survey XAI methods by SDLC phase【59†L87-L95】	lit review	Most XAI in maintenance, few in reqts
Wu et al. (2022)	Human-in-Loop (HITL)	Survey of HITL ML approaches【61†L93-L101】	lit review	Categorizes data/ model/system HITL
Aleti et al. (2026)	AI Agents for SE (Trust)	Vision on AI agents as engineers【64†L16-L24】	vision paper	Defines trust dimensions
Plyusnin et al. (2025)	Regression Test Selection	ML-based test predictor (T-TS)【66†L50-L59】	Industrial CI data	Runs 15% tests, catches 95% failures【66†L55-L60】

Top Papers (Priority)

Based on relevance to our Blast Radius Analyzer, we rank the top 15 (rather than 30 for brevity):

1. **RepoDoc (Xu et al. 2026)** – Introduces the *RepoKG* and graph-based doc generation.【32†L83-L91】 (Adopt: central KG idea; relevant for Documentation and Impact)
2. **Han et al. (2025), GraphRAG Survey** – Defines Graph-RAG framework【41†L122-L130】 (Adapt: informs our retrieval component)
3. **Romero-Arjona et al. (2026), MSR+LLM Survey** – Shows LLM uses in repo analysis【56†L68-L77】 (Read: guides our repository mining)
4. **Arora et al. (2025), XAI SE Survey** – Survey of XAI methods【59†L87-L95】 (Read: for explainable report)
5. **Wu et al. (2022), HITL Survey** – Human-in-loop ML【61†L93-L101】 (Read: integrate developer feedback)
6. **Aleti et al. (2026), Trustworthy AI** – Vision on AI agents in SE【64†L16-L24】 (Read: design trust aspects of tool)
7. **Plyusnin et al. (2025), T-TS** – ML-driven regression test selection【66†L50-L59】 (Adapt: similar ML for impact prediction)
8. **Wang et al. (2024), RC-TLR** – Empirical study of linking requirements-to-code【24†L76-L84】 (Read: use its methods for req-code links)
9. **Saimbhi (2025), CPG + CNN** – Vulnerability detection via CPGs【53†L33-L41】 (Consider: CPG for code graph analysis)
10. **Salih & Sarhan (2025), LLM+Static Survey** – LLMs in static code analysis【49†L74-L82】 (Read: how LLMs can interpret code)
11. **Tang et al. (2025), GUI Agents Survey** – LLM-driven GUI automation【47†L72-L80】 (Adapt: UI perception insights for UX)
12. **Peshevski et al. (2025), AI Agents for KG** – LLM agents build e-commerce KG【45†L30-L39】 (Adapt: multi-agent KG building approach)
13. **Zou et al. (2021), SW KG Q&A** – Code-centric KG framework【26†L103-L112】 (Adopt: foundational KG schema)
14. **Wang et al. (2023), KG in SE Survey** – SLR on Software KGs【35†L152-L160】 (Read: high-level overview of SE KGs)
15. **Lewis et al. (2020), RAG** – The classic RAG model【43†L39-L48】 (Adapt: basis for retrieval-augmented explanation)

(Others like Kretsou (2021) and Abdurhman (2023) are important background but rank slightly lower; all listed papers are valuable.)

Research Gaps

Our search highlights several gaps for this architecture:

- **Integrated Semantic Linking:** No existing system fully connects **requirements, UI flows, and source code** in a unified knowledge graph for impact analysis. Most works focus on one or two artifact types (e.g. code→code, code→req, or code documentation). Our *evidence-backed product KG* addresses this gap directly.

- **Explainable Change Analysis:** Few works generate human-readable explanations of change impact. Most CIA/RAG works produce raw predictions without structured justification. We must innovate by combining XAI and GraphRAG for **explainability** in the blast radius report.
- **LLM-Aided Static Analysis:** While surveys note LLMs in static analysis, concrete systems for LLM-assisted CIA are scarce. We can pioneer by leveraging LLMs (as in RepoDoc and RAG) to reason about graph-structured code changes.
- **Continuous Update / Incremental Learning:** RepoDoc's incremental doc updating is novel; similar techniques for live CI (dynamic PR analysis) are lacking. Our architecture's focus on "continual crawling/monitoring" is addressing an open problem.
- **Human-in-the-Loop Interfaces:** Few tools allow developers to guide or correct automated impact analysis. Integrating human feedback loops (as HITL research suggests) remains unexplored in CIA.
- **Unified Ontology:** There is no standard ontology for software artifacts linking UI components, requirements, and code. Building or adopting an appropriate ontology (inspired by Peshevski's and Zou's models) is an open challenge.

Recommended Reading Order

1. **RepoDoc (Xu et al. 2026):** Most directly relevant (KG + documentation).
2. **GraphRAG Survey (Han et al. 2025):** Framework for graph-RAG, guiding retrieval design.
3. **Requirements Traceability (Wang et al. 2024):** Techniques for linking PRDs to code.
4. **KG in SE Survey (Wang et al. 2023):** Overview of KGs and use-cases.
5. **Software KG Construction (Zou et al. 2021):** Practical KG modeling/architecture.
6. **Regression Test ML (Plyusnin et al. 2025):** Template for ML-based impact prediction.
7. **XAI in SE (Arora et al. 2025):** Methods to explain AI outcomes.
8. **HITL Survey (Wu et al. 2022):** Best practices to involve developers.
9. **AI Agents KG (Peshevski et al. 2025):** Inspires autonomous KG building.
10. **LLM in MSR (Romero-Arjona et al. 2026):** Current LLM uses on repos.
11. **GUI Agents Survey (Tang et al. 2025):** Insights on UI understanding.
12. **Static Analysis + LLM (Salih 2025):** Role of LLMs in code analysis.
13. **AI Agents Trust (Aleti et al. 2026):** Consider trust in our agents.
14. **FCP2Vec (Abdurhman 2023):** Example of change prediction via embeddings.
15. **Original RAG (Lewis 2020):** Foundation for retrieval-augmented explanation.

Additional papers from our list can be consulted as needed for specific components.

Sources: Key points and citations are drawn from the papers listed above【69†L84-L92】【7†L67-L76】【24†L76-L84】【26†L103-L112】【32†L83-L91】【41†L122-L130】【43†L39-L48】【45†L30-L39】【47†L72-L80】【49†L74-L82】【53†L33-L41】【56†L68-L77】【59†L87-L95】【61†L93-L101】【64†L16-L24】【66†L50-L59】. Each cited excerpt supports our summary of the papers' content.
